

1. 가상스레드 전 개념 다지기

이 글은 공식 문서와 크롬 확장 프로그램(Claude)을 통해 번역·학습한 내용입니다. 100% 정확하지 않을 수 있으니 이 점 감안해 주세요.

가상 스레드로 넘어가기 전에, 그 토대가 되는 개념들(프로세스와 스레드의 차이, Executor, Runnable / Callable / Future)을 먼저 정리한다.

1. Process vs Thread

- **Process**: 자기만의 메모리 공간을 가진다.
- **Thread**: 프로세스 안에 존재하며, 프로세스의 자원을 공유한다.
- 공식 문서에서는 스레드를 **경량 프로세스(lightweight process)** 라고 부른다.

새 스레드를 만드는 것은 새 프로세스를 만드는 것보다 자원을 덜 요구한다. 다만 스레드가 프로세스보다 가볍다는 것이지, 여전히 자원을 소모하는 존재라는 점은 같다.

모든 애플리케이션은 최소 하나의 스레드로 시작하고, 필요하면 추가 스레드를 만들 수 있다.

2. Thread 객체 — 스레드를 다루는 두 가지 전략

Oracle 튜토리얼에 따르면, 작은 애플리케이션에서는 **Runnable** 과 **Thread** 를 직접 묶어도 괜찮지만, 대규모 애플리케이션에서는 스레드의 생성·관리를 코드에서 분리하는 것이 합리적이다. 이 기능을 캡슐화한 객체를 **executor**라고 부른다.

각 스레드는 **Thread** 클래스의 인스턴스와 연결된다.

- **전략 1**: **Thread** 를 직접 인스턴스화해, 스레드의 생성과 관리를 직접 제어한다.
- **전략 2**: 애플리케이션의 task를 **executor** 에게 넘겨, 스레드 관리를 코드로부터 분리한다.

가상 스레드(Virtual Thread)는 기존 **Thread API** 를 그대로 재사용한다. 그리고 작업을 **executor** 에게 위임하는 방식(전략 2)을 사용한다.

3. Executor와 ExecutorService — 작업 관리 추상화

java.util.concurrent 패키지는 세 가지 인터페이스를 정의한다.

- **Executor**: 새 작업을 실행하는 단순한 인터페이스.
- **ExecutorService**: Executor의 하위 인터페이스. 개별 작업과 executor 자체의 생명주기 관리 기능이 추가되어 있다.
- **ScheduledExecutorService**: ExecutorService의 하위 인터페이스. 미래 시점 실행, 주기적 실행을 지원한다.

Executor
└─ ExecutorService

`Executor` 인터페이스는 `execute()` 메서드 하나를 제공한다. 이 안에서 새 스레드를 직접 만들어 실행할 수도 있지만, 실제로 `executor`의 `execute()`는 기존 **worker thread**를 재사용하거나, **worker thread**가 사용 가능해질 때까지 작업을 큐에 넣을 가능성이 더 높다.

4. Callable과 Future — 결과가 있는 작업

`ExecutorService`는 `execute()`보다 더 유연한 `submit()` 메서드를 제공한다.

`submit()`은 `Runnable` 뿐 아니라, 값을 반환할 수 있는 `Callable` 객체도 받는다. 그리고 `Future` 객체를 반환하는데, 이 `Future`는 `Callable`의 반환 값을 가져오고 `Callable` 및 `Runnable`의 상태를 관리하는 데 사용된다.

결과를 취합할 때 `Future.get()`으로 결과를 받는데, 이때 **블로킹**이 발생한다.

5. Runnable / Callable / Future 자세히

5.1 Runnable

Runnable: 결과를 반환하지 않는 작업.

`java.lang.Runnable`은 **함수형 인터페이스**이며, 공식 문서에서 "결과를 반환하지 않는 연산"을 나타낸다고 정의한다. 함수형 메서드는 `run()` 하나뿐이고, 반환 값이 없다.

```
void run();
```

5.2 Callable

Callable: 결과를 반환하고, 예외도 던질 수 있는 작업.

`java.util.concurrent.Callable<V>`도 **함수형 인터페이스**이며, "결과를 반환하며 예외를 던질 수 있는 작업"을 나타낸다. 함수형 메서드는 `call()`이고, 타입 파라미터 `V`가 반환 타입이다.

`Runnable`은 결과를 반환하지 못하고 (검사) 예외도 던지지 못하지만, `Callable`은 이 두 제약을 모두 해결한다. 즉, **반환 값이 필요하거나 예외를 던져야 할 때** 사용한다.

5.3 Future

Future: 비동기 계산의 결과를 담는다.

`java.util.concurrent.Future<V>`는 비동기 계산의 결과를 나타낸다. 이 인터페이스는 계산이 끝났는지 확인하고, 완료를 기다리고, 결과를 가져오는 메서드를 제공한다.

결과는 `get()` 메서드로 가져오며, **계산이 완료될 때까지 필요하면 블로킹**한다. 즉, `future.get()`을 호출했는데 아직 작업이 안 끝났다면, 그 스레드는 결과가 준비될 때까지 멈춰서 기다린다.

- `get()`: 완료를 기다린 뒤 결과를 반환한다.

- `get(long timeout, TimeUnit unit)` : 최대 지정 시간만 기다린다.
- `cancel(boolean)` : 취소를 시도한다.
- `isDone()` : 완료 여부를 알려준다.
- `isCancelled()` : 취소 여부를 알려준다.

Java 19에서 `resultNow()`, `exceptionNow()`, `state()` 메서드가 추가되었다.

참고 자료

dev.java (최신 릴리스 기반 튜토리얼)

- **Learn Java (학습 허브)** — <https://dev.java/learn/> 자바 기초부터 컬렉션, 스트림, 가상 스레드까지 주제별 튜토리얼 목록이 정리된 전체 학습 목차 페이지. 로드맵을 짤 때 선행 주제를 파악하려고 참고했다.
- **Virtual Threads** — <https://dev.java/learn/new-features/virtual-threads/> 이번 학습의 핵심 튜토리얼. Cay Horstmann이 쓴 글로, 가상 스레드가 왜 필요한지, 생성 방법, Thread API 변화, 결과 취합, Rate Limiting, Pinning, ThreadLocal 주의점까지 실용적으로 다룬다.

docs.oracle.com — Oracle 공식 튜토리얼 (The Java™ Tutorials, JDK 8 기준 기초)

- **Processes and Threads** — <https://docs.oracle.com/javase/tutorial/essential/concurrency/procthread.html> 프로세스와 스레드의 개념 차이(독립 메모리 공간, 경량 프로세스로서의 스레드)를 설명하는 페이지.
- **Thread Objects** — <https://docs.oracle.com/javase/tutorial/essential/concurrency/threads.html> Thread 객체를 다루는 두 가지 전략(직접 생성 vs executor 위임)을 소개하는 페이지.
- **Executors** — <https://docs.oracle.com/javase/tutorial/essential/concurrency/executors.html> executor의 개념(스레드 생성·관리를 애플리케이션에서 분리)을 개괄하는 페이지.
- **Executor Interfaces** — <https://docs.oracle.com/javase/tutorial/essential/concurrency/exinter.html> Executor, ExecutorService, ScheduledExecutorService 세 인터페이스와 submit / Future / Callable의 관계를 설명하는 페이지.

docs.oracle.com — Oracle 공식 API 문서 (Javadoc, Java SE 21)

- **Runnable** — <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Runnable.html> "결과를 반환하지 않는 작업"인 Runnable 인터페이스의 공식 정의 (`run()` 메서드).
- **Callable** — <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Callable.html> "결과를 반환하고 예외도 던질 수 있는 작업"인 Callable<V> 인

터페이스(`call()` 메서드)의 공식 정의.

- **Future** — <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Future.html> 비동기 계산 결과를 나타내는 `Future<V>` 인터페이스 페이지. `get()`의 블로킹 동작, `cancel()`, `isDone()` 등의 메서드와 공식 예제(`ArchiveSearcher`)가 실려 있다.
- **Thread** — <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html> `Thread` 클래스의 공식 API 문서. `ofVirtual()`, `startVirtualThread()`, `isVirtual()`, `threadId()`, `setDaemon()` (가상 스레드는 항상 daemon) 등의 정의를 확인하는 데 사용.

docs.oracle.com — Oracle 공식 가이드 (Java SE 21 Core Libraries)

- **Virtual Threads (Core Libraries)** — <https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html> Oracle의 공식 가상 스레드 가이드. 플랫폼 스레드 vs 가상 스레드 정의, 생성/실행 예제(에코 서버 포함), carrier / mount / unmount 스케줄링 원리, Pinning, 디버깅 (`JFR-jcmd`), 채택 가이드(폴링 금지·`Semaphore.ThreadLocal`)까지 규범적으로 다룬다.